DEVELOPER REFERENCE v2.0

Dynamic Data Engine

Architecture & Implementation Guide

Metadata-Driven CRUD Engine for Multi-Database Enterprise Systems

A comprehensive reference covering TransactionService, FieldMapper, ValidationService, AutoNumberService, DataTypeConverter, OperationScope, multi-provider support (PostgreSQL / MySQL / SQL Server / Oracle), audit logging, child table persistence, and React frontend integration.

Document Type	Architecture + Implementation Reference
Version	2.0 — Enhanced Edition
Classification	Internal Engineering · Confidential
Scope	Backend .NET 8 DLL + React 18 Frontend Suite
Providers	PostgreSQL · MySQL · SQL Server · Oracle

Table of Contents

1 Executive Summary & Design Philosophy

What this engine is and why it exists

This document describes a **metadata-driven, multi-provider CRUD engine** designed to eliminate per-table repository boilerplate in enterprise applications. Rather than writing a Service, Repository, and DTO for every database table, this engine accepts a structured JSON request, resolves the target table from a central registry, and dynamically generates fully parameterised SQL at runtime.

The engine is built on four foundational pillars drawn from production enterprise data platforms and enhanced with modern .NET 8 patterns:

METADATA-DRIVEN

FieldMapper metadata governs every table's columns, types, PK, required flags, and child relationships — no hardcoded schema anywhere in the engine.

MULTI-PROVIDER

A SqlDialect abstraction layer covers PostgreSQL, MySQL, SQL Server, and Oracle — parameter syntax, pagination, and returning-clause differences are encapsulated per dialect.

TRANSACTIONAL

OperationScope wraps every write in a database transaction. Parent inserts, all child inserts, and audit writes commit atomically or roll back together.

OBSERVABLE

AuditService captures before/after snapshots on every update and delete. AutoNumberService generates domain-specific keys. ValidationService enforces rules before any SQL is constructed.

Core Responsibility in One Sentence: Accept a structured transaction request, resolve all metadata from the registry, validate the payload, generate safe parameterised SQL, execute atomically with parent-child support, capture an audit trail, and return a typed result.

Design Philosophy

Principle	Implementation Decision
Generic over specific	One TransactionService handles any registered table — no table-specific code
Registry as truth	FieldMapper metadata in the database is the single source for all schema knowledge
Safe by default	All values are Dapper parameters; column/table names validated against registry whitelist
Atomic always	OperationScope ensures parent + children commit or rollback as one unit
Dialect-agnostic	SqlDialect value type isolates all provider differences from business logic
Auditable	Every update and delete captures old/new state before commit
Observable	Serilog structured events on every operation with entity, provider, duration

2 Architecture Overview

Component layers and system map

The engine is structured into five distinct layers, each with a single responsibility. No layer reaches past its immediate neighbour.

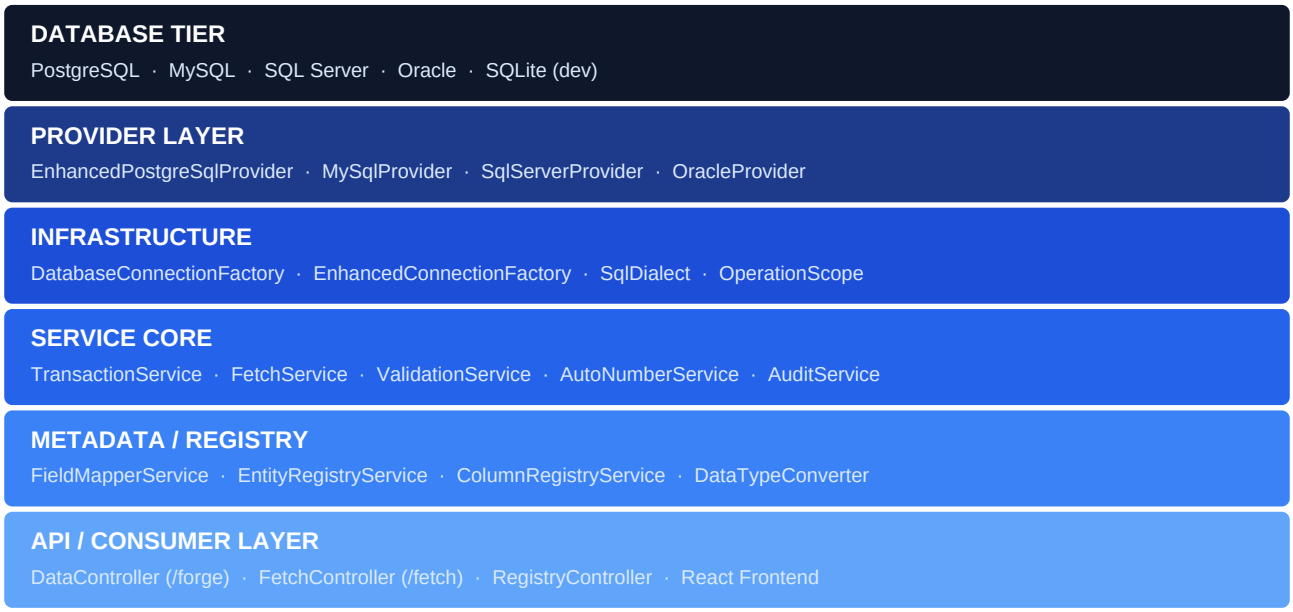


Figure 1 — Architecture layer stack (bottom = database, top = consumer)

2.1 Full System Component Map

2.2 Multi-Database Provider Architecture

Every provider implements **IEnhancedDataProvider**, a contract that abstracts all database execution. The SqlDialect value type carries provider-specific SQL syntax differences so that TransactionService and FetchService never branch on provider type.

Dialect Concern	PostgreSQL	MySQL	SQL Server	Oracle
Param prefix	@col	@col	@col	:col
Pagination	LIMIT n OFFSET s	LIMIT n OFFSET s	OFFSET s ROWS FETCH NEXT n	OFFSET s ROWS FETCH NEXT n
Returning new ID	RETURNING id	SELECT LAST_INSERT_ID()	SELECT SCOPE_IDENTITY()	RETURNING id INTO :newId
Schema separator	"schema"."table"	`schema`.`table`	[schema].[table]	SCHEMA.TABLE
Bool type	BOOLEAN	TINYINT(1)	BIT	NUMBER(1)
UUID type	UUID	CHAR(36)	UNIQUEIDENTIFIER	VARCHAR2(36)
Current time	NOW()	NOW()	GETUTCDATE()	SYSDATE

3 Core Data Models & Contracts

Request/response shapes and FieldMapper metadata

3.1 Transaction Payload — /forge (Write Operations)

The forge payload is the universal write contract. It handles create, update, and soft-delete for any registered entity, including nested child records in one atomic call.

3.2 Fetch Payload — /fetch (Read Operations)

3.3 FieldMapper Metadata Model

FieldMapper rows are stored in the registry database. They define every aspect of a table's behaviour — the engine reads them at runtime to drive all SQL generation, validation, type conversion, and child relationship handling.

4 Registry & Metadata System

EntityRegistry, ColumnRegistry, DbProfiles

4.1 Registry Database Schema

The registry is a dedicated metadata database (typically lightweight PostgreSQL or SQLite) that is entirely separate from any business database. It is the engine's single source of truth for all schema knowledge.

DbProfiles Table

Column	Type	Description
profile_id	UUID PK	Unique profile identifier
profile_name	VARCHAR(100)	Human name e.g. PostgresERP, MySqlHR
provider	VARCHAR(20)	postgresql | mysql | sqlserver | oracle | sqlite
host	VARCHAR(255)	Server hostname or IP
port	INT	Default: PG 5432, MySQL 3306, MSSQL 1433, Oracle 1521
database_name	VARCHAR(255)	DB name or SQLite file path
username	VARCHAR(100)	Connection username
password_hash	VARCHAR(500)	AES-256-CBC encrypted — never plaintext
status	VARCHAR(20)	active | inactive | error
created_at	TIMESTAMPTZ	UTC creation timestamp

EntityRegistry Table

Column	Type	Description
entity_id	UUID PK	Unique entity identifier
profile_id	UUID FK	FK → DbProfiles.profile_id
entity_name	VARCHAR(200)	Exact DB table name (e.g. purchase_order)
schema_name	VARCHAR(100)	Schema prefix (e.g. public, dbo, JANATICS)
pk_column	VARCHAR(100)	Primary key column name (e.g. id)
pk_type	VARCHAR(20)	uuid | autoincrement | autonumber
auto_number_key	VARCHAR(100)	AutoNumber config key if pk_type=autonumber
soft_delete_col	VARCHAR(100)	Column to flag on delete (e.g. is_deleted)
allowed_roles	VARCHAR(500)	Comma-separated roles for access control
is_readonly	BOOLEAN	Block all forge operations
created_by	VARCHAR(100)	Admin who registered this entity

ColumnRegistry Table

Column	Type	Description
column_id	UUID PK	Unique column identifier
entity_id	UUID FK	FK → EntityRegistry.entity_id (CASCADE DELETE)
column_name	VARCHAR(200)	Exact DB column name
field_name	VARCHAR(200)	API/UI field name (used in request Content dict)
data_type	VARCHAR(50)	text | int | decimal | bool | date | uuid | json
is_required	BOOLEAN	Validated on forge create operations
is_pk	BOOLEAN	True if this is the primary key
is_readonly	BOOLEAN	Excluded from INSERT and UPDATE
is_audit	BOOLEAN	Auto-set by engine (created_at, modified_by etc.)
is_fk	BOOLEAN	References another entity
fk_reference	VARCHAR(300)	entity_name.column_name (e.g. vendor.vendor_id)
max_length	INT	Max character length for validation
child_entity	VARCHAR(200)	Child entity name if this column is a relationship
parent_link	VARCHAR(200)	FK column on child table pointing back to parent

4.2 EntityRegistryService — Cache & Lookup

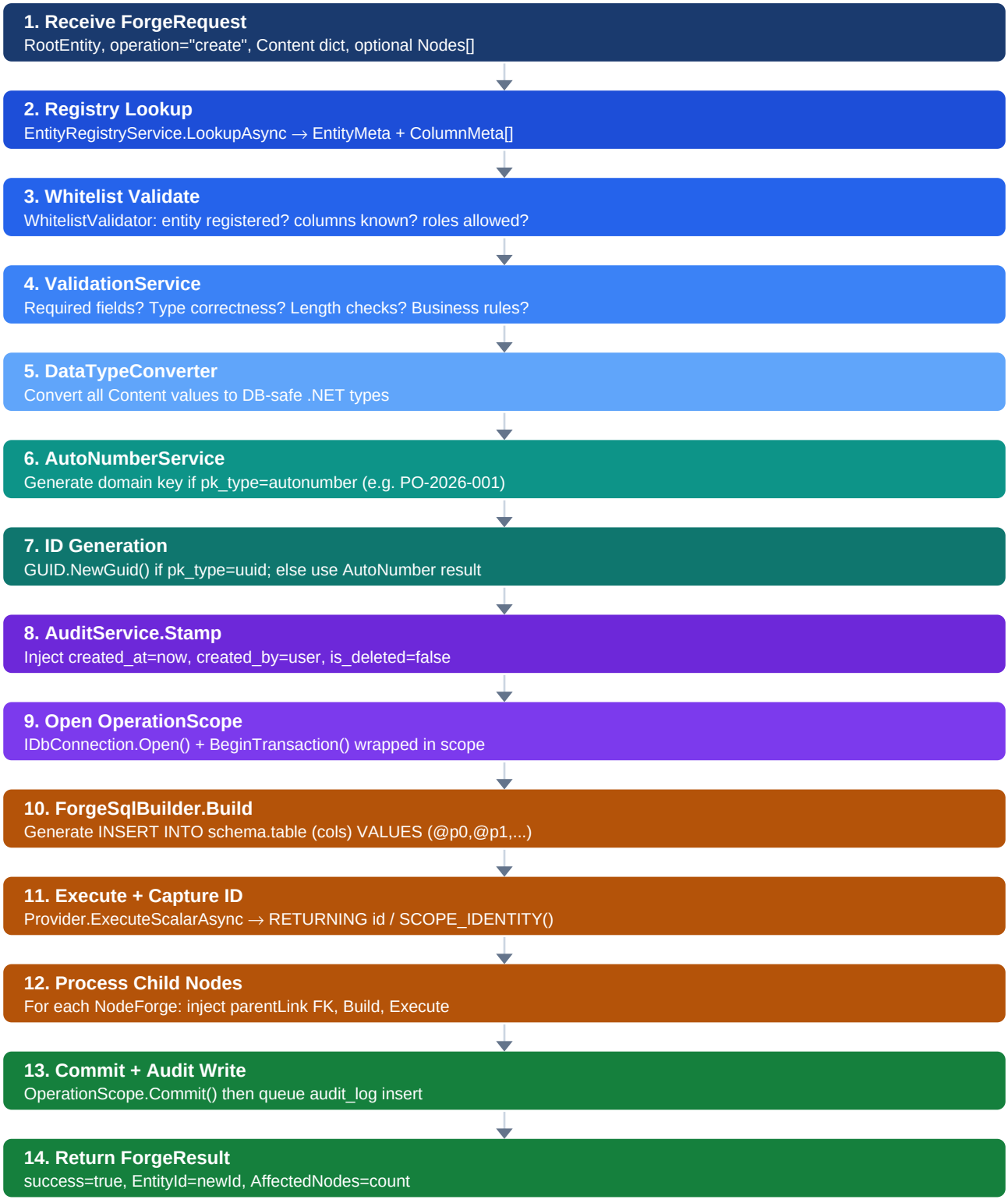
The registry service wraps all MetaDB queries with an **IMemoryCache** layer. Cache keys are namespaced by entity name and profile ID. TTL is configurable (default 5 minutes). Admin updates via the Registry Manager UI call an invalidation endpoint that removes affected cache entries immediately.

5 TransactionService — CRUD Orchestrator

The central engine for all create, update, and delete operations

TransactionService is the heart of the engine. It receives a ForgeRequest, orchestrates all supporting services, and returns a ForgeResult. It never generates SQL directly — that responsibility belongs to ForgeSqlBuilder. It never validates directly — that belongs to ValidationService. It is purely an orchestrator.

5.1 Insert (Create) Flow



5.2 Update Flow

6 FieldMapper & Dynamic SQL Builder

How metadata drives safe parameterised SQL generation

6.1 FetchSqlBuilder

FetchSqlBuilder is a pure static function. Same inputs always produce same SQL. It operates in six stages and always uses the SqlDialect abstraction — it never branches on provider type directly.

6.2 ForgeSqlBuilder

7 ValidationService

Pre-save data validation driven by ColumnRegistry metadata

ValidationService runs before any SQL is built. It reads validation rules from the ColumnRegistry (required, maxLength, dataType) and can be extended with business rule plugins per entity.

Validation Type	Source	Behaviour on Failure
Required field check	ColumnRegistry.is_required	Adds field to error list — "FieldName is required"
Data type check	ColumnRegistry.data_type	Attempts conversion; on failure adds type mismatch error
Max length check	ColumnRegistry.max_length	Compares string length; adds length error if exceeded
Read-only field check	ColumnRegistry.is_readonly	Removes field from content — does not error
Unknown field check	ColumnRegistry whitelist	Throws SqlBuildException — unknown column is security risk
Business rule plugin	IValidationRule per entity	Plugin returns ValidationResult with custom message

8 AutoNumberService

Domain-specific key generation with configurable patterns

AutoNumberService generates human-readable sequential identifiers for entities that require domain keys (invoices, purchase orders, employee codes) rather than raw UUIDs. It uses a dedicated **auto_number_config** table with atomic sequence increments to prevent duplicates under concurrent load.

auto_number_config Table

Column	Type	Example	Description
config_key	VARCHAR(100) PK	purchase_order	Entity/use-case identifier
prefix	VARCHAR(20)	PO-	Static prefix string
year_segment	BOOLEAN	true	Append current year (2026)
separator	VARCHAR(5)	-	Separator between segments
current_seq	BIGINT	123	Current sequence value (atomic++)
seq_length	INT	4	Zero-padded length: 0123
reset_yearly	BOOLEAN	true	Reset sequence on year change
last_reset_yr	INT	2026	Year of last reset

▶ Generated Key Examples

PO-2026-0123 (purchase_order) | INV-2026-00045 (invoice) | EMP-1044 (employee) | SO202600123 (sales_order)

9 DataTypeConverter

Safe runtime conversion of request values to DB-typed .NET objects

All values arriving in **Content** dictionaries are untyped (object?). DataTypeConverter maps each value to the correct .NET type based on the ColumnRegistry data_type before SQL parameters are bound.

data_type	Input Examples	.NET Target	Edge Cases
text	"hello", 123	string	null → null; numbers toString
int	"42", 42, "042"	int	"" or null → null if not required
decimal	"12.50", 12.5, "12,50"	decimal	Locale comma handling
bool	"true", "1", "yes", true	bool	Case-insensitive; "0"/"false"/"no" → false
date	"2026-04-29", "29/04/26"	DateTime (UTC)	Multiple ISO + locale formats
uuid	"550e8400-e29b..."	Guid	Invalid format throws ConversionException
json	"{...}"	string (raw)	Stored as text; no parsing by engine

10 OperationScope & Transaction Safety

Atomic parent-child persistence with automatic rollback

OperationScope wraps every forge operation in a database transaction. It is disposable — if **CommitAsync()** is never called before disposal, the transaction is automatically rolled back. This ensures parent + all child records either all succeed or none persist.

⚠ Critical Safety Rule

OperationScope must always be used with "await using" (C# async dispose pattern). Never call CommitAsync() inside a try block that swallows exceptions — let the using scope handle rollback on any unhandled exception.

11 AuditService

Before/after diff capture for updates and deletes

AuditService writes to an **audit_log** table after every successful forge operation. For creates it stores the new state. For updates it stores an old/new diff. For deletes it stores the final state before soft-deletion. Audit writes happen *after* CommitAsync() so they never block the main transaction.

Column	Type	Description
log_id	UUID PK	Audit entry identifier
entity_name	VARCHAR(200)	Table name (e.g. purchase_order)
entity_id	VARCHAR(100)	PK value of the affected row
operation	VARCHAR(20)	create | update | delete
changed_by	VARCHAR(200)	Username from ClaimsPrincipal
changed_at	TIMESTAMPTZ	UTC timestamp
old_state	JSONB / TEXT	Row state before update/delete (null for create)
new_state	JSONB / TEXT	Row state after create/update (null for delete)
diff	JSONB / TEXT	Key-level diff: {col: {old: x, new: y}}
ip_address	VARCHAR(50)	Request IP from HttpContext
session_id	VARCHAR(100)	Optional trace/correlation ID

12 ConnectionFactory & Pool Management

Multi-provider connection lifecycle and health checks

Two factory implementations serve different needs. **DatabaseConnectionFactory** provides fast, simple connection creation for CRUD writes. **EnhancedConnectionFactory** adds metrics, health checks, and read-replica routing for high-availability deployments.

Feature	DatabaseConnectionFactory	EnhancedConnectionFactory
Purpose	Standard CRUD writes	HA production deployments
Connection pool	ADO.NET default pool	Configurable min/max pool size
Health check	No	Periodic ping + status tracking
Read replica	No	Routes SELECT to replica by flag
Metrics	No	Connection acquire time, pool wait
Retry policy	None	Polly transient retry (3x, exp backoff)
Password handling	AES-256 decrypt at startup	Same + rotation support
Use case	Dev / staging / single-DB portals	Multi-DB production portals

13 WhitelistValidator & Security Model

Registry-enforced SQL injection prevention and access control

All security enforcement happens at the **WhitelistValidator** layer — before any SQL is constructed. This is the single chokepoint that prevents unknown table names, unknown column names, and unauthorised role access from reaching the SQL builder.

Threat Vector	Risk	Enforcement Point
Unknown entity name in request	CRITICAL	WhitelistValidator → EntityNotFoundException (404)
Unknown column name in Content	CRITICAL	WhitelistValidator → SqlBuildException (400)
Filter on unregistered column	HIGH	WhitelistValidator on each FilterCondition.Column
Filter value injection	HIGH	All values are Dapper @parameters — never concatenated
Write to read-only entity	HIGH	WhitelistValidator checks EntityMeta.IsReadOnly
Role-based table access	HIGH	AllowedRoles intersection vs ClaimsPrincipal.GetRoles()
Plaintext connection strings	HIGH	AES-256-CBC in DbProfiles; key in env var only
Mass data extraction	MEDIUM	PageSize capped at 1000; no unpaaginated export path
Audit bypass	LOW	AuditLog has no delete endpoint; append-only by design

14 FetchService — Read Engine

Paginated, filtered, searchable data retrieval

FetchService orchestrates all read operations. It is optimised for the most common ERP query pattern: paginated lists with filters, optional full-text search, optional child entity joins, and a total count for UI pagination controls.

15 API Endpoint Design

HTTP contracts for all engine capabilities

Method	Endpoint	Service	Description
POST	/api/forge	TransactionService	Create, update, or soft-delete any registered ent
POST	/api/fetch	FetchService	Paginated read with filters, search, sort, joins
GET	/api/fetch/{entity}/{id}	FetchService	Get single record by primary key
GET	/api/registry/profiles	RegistryController	List all DB profiles
POST	/api/registry/profiles	RegistryController	Register new DB profile
PUT	/api/registry/profiles/{id}	RegistryController	Update DB profile
DELETE	/api/registry/profiles/{id}	RegistryController	Remove DB profile
POST	/api/registry/profiles/{id}/test	RegistryController	Test DB connection, return latency
POST	/api/registry/profiles/{id}/discover	RegistryController	Introspect DB, return unregistered tables
POST	/api/registry/profiles/{id}/invalidate	RegistryController	Flush registry cache for profile
GET	/api/registry/entities	RegistryController	List entities for a profile
POST	/api/registry/entities	RegistryController	Register new entity
PUT	/api/registry/entities/{id}	RegistryController	Update entity metadata
DELETE	/api/registry/entities/{id}	RegistryController	Remove entity from registry
GET	/api/registry/entities/{id}/columns	RegistryController	List columns for entity
POST	/api/registry/entities/{id}/columns	RegistryController	Add column to entity
DELETE	/api/registry/columns/{id}	RegistryController	Remove column from registry
GET	/api/autonumber/configs	AutoNumberController	List auto-number configurations
GET	/api/audit/{entity}/{id}	AuditController	Get audit history for a record

Unified Error Response Envelope

16

React Frontend Suite

Registry Manager · Query Builder · Field Mapper

The frontend is a React 18 + TypeScript admin suite built with Ant Design 5.x and TanStack Query v5. All three pages are driven by the registry API — they never hardcode table or column names.

Page	Route	Primary Purpose	Calls
Registry Manager	/admin/registry	Admin: manage DB profiles, entities, columns, auto-numbering rules	/api/registry/*
Query Builder	/admin/query	Developer: visual /fetch query composer with live results	/api/fetch
Field Mapper	/admin/forge	User: visual /forge payload builder for create/update operations	/api/forge

Frontend Component Tree

▶ TanStack Query Pattern

All API calls use useMutation (forge) and useQuery (fetch/registry). Registry data is cached with staleTime=5min matching backend CacheTtlMinutes. Cache invalidation after registry mutations uses queryClient.invalidateQueries.

17 Error Handling Strategy

Exception hierarchy, middleware, and result types

18 Testing Strategy

Unit, integration, and contract test coverage

Layer	Framework	Scope	Target Count
Unit	xUnit + Moq	SqlBuilders (all operators), DataTypeConverter, AutoNumberService, WhitelistValidator, ValidationSe	~140 tests
Integration	xUnit + SQLite in-memory	FetchService, TransactionService, AuditService against real schema	~50 tests
Provider	xUnit + Testcontainers	Dialect SQL output verified against real PostgreSQL/MySQL Docker containers	~30 tests
Contract	Pact.NET	Consumer-driven contracts for /forge and /fetch request schemas	~20 tests
E2E	Playwright	Registry Manager + Query Builder + Field Mapper against staging	~15 tests

Key Unit Test Cases

- FetchSqlBuilder: "between" operator produces BETWEEN @p0 AND @p0b with both parameter values
- FetchSqlBuilder: "contains" wraps value in %...% and uses LOWER() for case-insensitive match
- FetchSqlBuilder: pagination clause uses LIMIT/OFFSET for PostgreSQL/MySQL vs OFFSET ROWS FETCH NEXT for SQL Server/Oracle
- ForgeSqlBuilder: INSERT excludes is_readonly columns; UPDATE excludes PK and readonly columns
- ForgeSqlBuilder: soft-delete produces UPDATE SET is_deleted=true — never DELETE FROM
- DataTypeConverter: "true", "1", "yes", "on" all convert to boolean true
- DataTypeConverter: empty string converts to null for nullable fields
- AutoNumberService: concurrent GenerateAsync calls never produce duplicates (using FOR UPDATE lock)
- ValidationService: required field missing on create produces error; missing on update is allowed
- WhitelistValidator: unknown column name throws SqlBuildException before any SQL is built
- OperationScope: uncommitted scope disposes with rollback — no partial inserts persist

19 Enhancement Roadmap

Priority improvements identified from reference architecture analysis

The following improvements are prioritised based on the reference architecture analysis, targeting the known risks in dynamic SQL engines and unlocking enterprise-scale capabilities.

Phase 1 — Security & Stability (Weeks 1–2)

- **Replace any string SQL concatenation with ForgeSqlBuilder parameterised output** — audit all provider implementations for raw string interpolation in SQL
- **Add ColumnRegistry whitelist check for every filter column** in /fetch requests — WhitelistValidator.ValidateFetch must cover FilterCondition.Column
- **AES-256-CBC for all DbProfile password fields** — add migration script for existing plaintext entries
- **Add pageSize cap of 1000** to FetchService — no path should allow unpaaginated full-table reads

Phase 2 — Performance (Weeks 3–4)

- **Cache FieldMapper metadata in IMemoryCache** with 5-minute TTL — eliminate repeated MetaDB reads per request
- **Add bulk insert path** to ForgeSqlBuilder — for Nodes[] with 50+ child records, generate VALUES (...),(...),... instead of N individual INSERT statements
- **Instrument every Dapper call with Stopwatch** — log Warning if execution >500ms, Error if >2000ms
- **Add Serilog structured events** on every forge/fetch with entity, provider, rowCount, executionMs

Phase 3 — Capability (Weeks 5–8)

- **Soft-delete strategy** — configure SoftDeleteColumn/SoftDeleteValue per entity in EntityRegistry; all "delete" operations set this flag rather than DELETE FROM
- **Multi-provider Testcontainers integration tests** — verify dialect SQL against real PostgreSQL and MySQL Docker containers in CI
- **Business rule plugin system** — IValidationRule interface loadable per entity from DI container
- **Bulk import endpoint** — POST /api/forge/bulk accepts array of ForgeRequests in single transaction
- **Auto-discover enhancement** — introspect INFORMATION_SCHEMA to auto-populate ColumnRegistry on import, inferring data types from DB column types

Architecture Decision Summary

Decision	Recommendation	Rationale
ORM vs Dapper	Keep Dapper	Dynamic SQL generated at runtime cannot be modelled with EF Core's static DbSet<
Registry storage	MetaDB (dedicated)	Runtime mutability, UI governance, FK integrity, no redeploy for new entity
Soft delete	Always soft	Manufacturing ERP data is legally auditable; physical deletes break audit chains
Auto number lock	SELECT FOR UPDATE row lock	Prevents duplicate keys under concurrent load at cost of brief row contention
Audit write timing	Post-commit fire-and-forget	Audit failures must never roll back business data
Cache invalidation	Explicit + TTL hybrid	Admin endpoint flushes immediately; TTL acts as safety net